

1. Planejamento da capacitação de Git e GitHub

- Realização em 2 dias, 1:00 a 1:30 de duração;
- Abordar conceitos sobre Git e GitHub.

1.1. Conteúdo programado

Dia	Capítulo	Conteúdo
1º	Introdução	Introdução
1º	Git	Instalação
1º	Git	Comandos básicos + Fluxo de trabalho
1º	GitHub	Overview + Comandos de repositório remoto
2º	GitHub	Intermediário
2º	Git + GitHub	Intermediário

1.1.1. Introdução

Explicação sobre o Git, plataformas de hospedagem de repositórios e suas diferenças.

1.1.2. Instalação

Instalação do Git em ambiente Windows, Linux e MacOS.

1.1.3. Comandos básicos + Fluxo de trabalho

Apresentação dos comandos `init`, `config`, `add`, `commit`, `status`, `log` e `checkout`; Explicação sobre o fluxo de trabalho do Git.

1.1.4. Overview da plataforma GitHub + Comandos de repositório remoto

Apresentação da plataforma; Criação de repositórios; Comandos `clone`, `push` e `pull`.

1.1.5. GitHub intermediário

Abordagem sobre o uso da ferramenta *Projects*; *Issues*; *Review* e *Pull Requests*; Configuração do projeto;

1.1.6. Git + GitHub intermediário

Conceito de *branches*; Conceito de *merge*; Conceito de *rebase*; Ocorrência de conflitos e correções; Desfazer operações; .

Issues

Podem ser usadas para rastreio de bugs, adição de novas features, ser usado como histórico de alterações, ou seja, uma documentação. Hoje existe ainda a possibilidade você criar subissues para cada issue criada, possibilitando assim dividir partes maiores em trabalhos menores

Criação de issue

Algumas informações são importantes para a criação de uma issue sendo elas:

- Título: Resumo do que se trata a issue
- Assignees: Pessoas responsáveis por resolver a issue
- Descrição: Explicação detalhada da issue
- Rótulos: Ajudam a categorizar a issue, podendo ser desde um bug, a melhoria, uma dúvida entre outras
- Projetos: Vincula a issue a um projeto criado
- Milestones: Permite a adição de uma issue para controle baseado em datas
- Development: Permite vincular a issue a uma branch ou pull request.

Utilidades

- Quando estiver fazendo um commit é possível referenciar uma issue usando #<num_issue>, dessa forma temos como linkar um commit a uma issue, ou até mais de uma, o que é útil para uma rastreabilidade

Branches

O uso delas é outro grande ponto chave na escolha de uma ferramenta como o Git. A partir dela podemos fazer o desenvolvimento de diferentes partes do projeto de formas paralelas sem que uma parte necessariamente interfira na outra.

No uso do git é usado por padrão a branch master/main ou principal para indicar o estado mais estável do código fonte, ou seja, nessa branch reside todo um conjunto de arquivos consolidados e que podem ser utilizados sem problemas e o uso de uma branch diferente da master/main é uma boa prática para não afetar a parte principal, a não ser quando todas as alterações forem de fato consolidadas.

Mas como uma branch diferente sabe exatamente dos commits que já aconteceram? Isso ocorre dado que a cada commit realizado o Git mantém uma estrutura de árvore ligando cada commit a seu antecessor, dessa maneira ele consegue saber do histórico sem se perder da origem.

git branch

Para realizar a listagem das branches existentes no repositório podemos utilizar o comando `git branch`. A criação de branches pode ser realizada usando o comando `git branch <nome>`. Para mudar para uma branch é usando o comando `git checkout <nome>`

Esse processo de criação e mudança de branch pode ser feita usando apenas um comando `git checkout -b <nome>`



Outras formas de gerenciar branches:

- `git switch`: Comando mais recente e recomendado para criação e mudança de branch, pois é mais intuitivo e específico para essa função.

Branches remotas

Branches remotas são aquelas que estão hospedadas em um servidor, ou seja, não estão localmente. Para realizar a listagem de branches remotas podemos usar o comando `git branch -r`. Para criar uma branch remota é necessário primeiro criar a branch local e depois usar o comando `git push -u origin <nome>` para subir a branch para o repositório remoto. Visualize as branches remotas também usando `git branch -a` que mostra tanto as branches locais quanto as remotas.

- O origin é o nome do repositório remoto que foi configurado quando o repositório foi clonado, isso é possível ser visualizado ao usar o comando `git remote -v`, no caso dois serão listados, um para o fetch e outro para o push, ambos com a URL do repositório remoto.

O uso da flag `-u` significa que estamos adicionando um “upstream” para a branch, ou seja, estamos associando a branch local com a branch remota. Isso influencia em comando encurtados como `git push` e `git pull`, que sem a flag `-u` exigiriam a especificação do repositório remoto e da branch toda vez, mas com a flag `-u` eles já sabem para onde enviar ou de onde puxar as mudanças.

No GitHub é possível criar branches diretamente na plataforma, para isso basta acessar a página do repositório e clicar no menu de branches, depois clicar em “New branch” e escolher um nome para a nova branch. A partir disso a branch já estará disponível para ser utilizada localmente usando o comando `git checkout <nome>`.

Pull Requests e Reviews

No GitHub, um Pull Request (PR) é uma solicitação para mesclar mudanças de uma branch para outra, geralmente de uma branch com uma feature, correção de bug para a branch principal. Ele permite que os colaboradores revisem as mudanças propostas, discutam sobre elas e decidam se devem ser integradas ao projeto.

Configurações

- Título: Resumo do que se trata o PR
- Descrição: Explicação detalhada das mudanças propostas;
- Rótulos: Ajudam a categorizar o PR, podendo ser desde um bug, a melhoria, uma dúvida entre outras
- Projetos: Vincula o PR a um projeto criado
- Milestones: Permite a adição de um PR para controle baseado em datas;
- Assignees: Permite a atribuição de um PR para um colaborador específico;
- Reviewers: Permite a solicitação de revisão de um PR para um colaborador específico.

Revisão

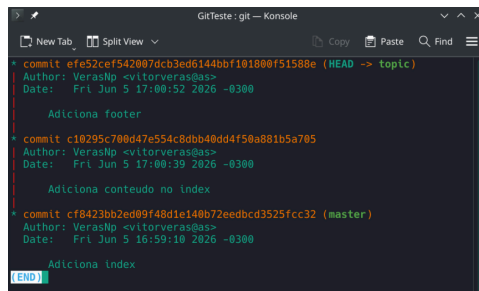
Na PR é possível observar desde os commits feitos, os arquivos modificados, esse ultimo é útil para os revisores entenderem o impacto das propostas, podendo assim fazer comentários específicos em linhas de código, ou seja, um comentário inline, ou um comentário geral sobre a PR.

O merge

Um processo de integração de branches, comum de ser realizado quando precisamos integrar uma branch de desenvolvimento para uma branch principal. Essa ação pode desencadear algumas particularidades que serão abordadas a seguir.

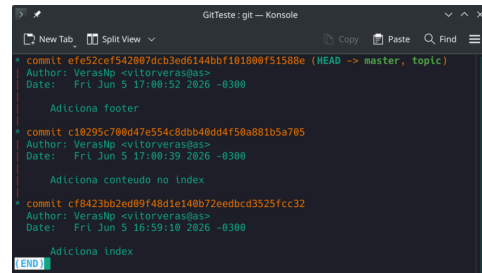
git merge

A partir de um commit A na branch master criamos uma branch topic e fazemos os commits B e C, ou seja, temos a seguinte configuração:



```
git log --graph
* commit efe52cef542007dcb3ed6144bbf101800f51588e (HEAD -> topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 17:00:52 2026 -0300
  Adiciona footer
* commit c10295c700d47e554c8dbb40dd4f50a881b5a705
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 17:00:39 2026 -0300
  Adiciona conteudo no index
* commit cf8423bb2ed09f48d1e140b72eedbcd3525fcc32 (master)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 16:59:10 2026 -0300
  Adiciona index
```

Para realizar o merge da branch topic para a branch master usando o comando `git merge topic` estando na branch master e teremos a seguinte configuração:



```
git log --graph
* commit efe52cef542007dcb3ed6144bbf101800f51588e (HEAD -> master, topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 17:00:52 2026 -0300
  Adiciona footer
* commit c10295c700d47e554c8dbb40dd4f50a881b5a705
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 17:00:39 2026 -0300
  Adiciona conteudo no index
* commit cf8423bb2ed09f48d1e140b72eedbcd3525fcc32
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 16:59:10 2026 -0300
  Adiciona index
```

Quando isso acontece podemos ter uma visão com `git log --graph` que ele trata como se a outra branch nunca existiu.

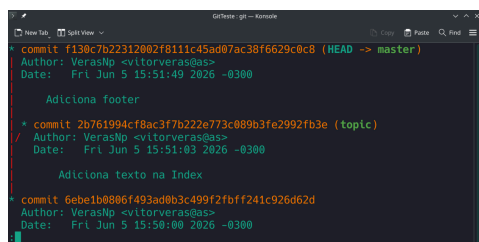
A integração ocorre sem que o git precise fazer grande esforço. Ocorre aqui um **Fast-Forward**.

Após a finalização do merge, a branch topic pode ser deletada usando o comando `git branch -d topic` ou `git branch -D topic` caso queira forçar a deleção mesmo que haja mudanças não integradas.

Mas nem tudo são flores, quando temos commits paralelos, ou seja, commits feitos na branch master e na branch topic a partir do commit A, o merge não é tão simples assim, ele gera um commit do tipo merge para indicar que houve um merge. O histórico fica mais complexo, mas ele preserva a estrutura de árvore dos commits, ainda sim é possível haver 2 possibilidades:

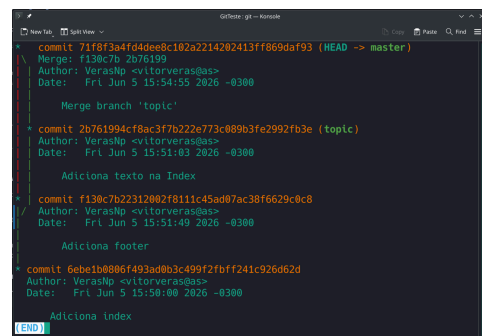
1. **Sem ocorrência de conflitos:** Não houve sobreposição de códigos, assim o Git faz o uso do que chamamos **Recursive Strategy**;

A partir de um commit A na branch master criamos uma branch topic e fazemos o commit B, depois na branch master realizamos um commit C, ou seja, temos a seguinte configuração:



```
git log --graph
* commit f130c7b22312002f811c45ad07ac38f6629c0c8 (HEAD -> master)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:49 2026 -0300
  Adiciona footer
* commit 2b761994cf8ac3f7b222e773c089b3fe2992fb3e (topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:03 2026 -0300
  Adiciona texto na index
* commit 6ebe1b0806f493ad0b3c499f2fbff241c926d62d
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:50:00 2026 -0300
```

Ao realizar o merge da branch topic para a branch master usando o comando `git merge topic` teremos a seguinte configuração:

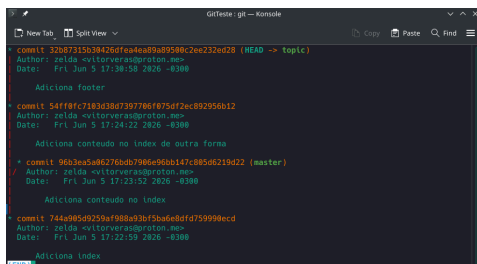


```
git log --graph
* commit 71f8f3a4fd4dee8c102a2214202413ff069daf93 (HEAD -> master)
  Merge: f130c7b 2b76199
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:54:55 2026 -0300
  Merge branch 'topic'
* commit 2b761994cf8ac3f7b222e773c089b3fe2992fb3e (topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:03 2026 -0300
  Adiciona texto na index
* commit f130c7b22312002f811c45ad07ac38f6629c0c8
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:49 2026 -0300
  Adiciona footer
* commit 6ebe1b0806f493ad0b3c499f2fbff241c926d62d
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:50:00 2026 -0300
  Adiciona index
```

Não houve assim conflito e o git fez a resolução de forma recursiva.

2. **Ocorrência de conflitos:** Houve sobreposição de códigos, assim é necessário fazer a resolução dos conflitos nos arquivos identificados de sobreposição e então depois gerar o commit de junção.

A partir de um commit A de uma arquivo `index.md` na branch `master` realizamos um commit B no mesmo arquivo, depois criamos uma branch `topic` e fazemos o commit C no mesmo arquivo `index.md` de antes e mais um commit D em outro arquivo `footer.md`, ou seja, temos a seguinte configuração:



```
GitTeste: git - Konsole
commit 320b7352b9426f1eade8b9d8580c2ee232e028 (HEAD -> topic)
Author: zelda <vitorveras@proton.me>
Date:   Fri Jun 5 17:24:22 2026 -0300
    Adiciona footer

commit 54ff0fc7183d38d7397786f075af2ec892950b12
Author: zelda <vitorveras@proton.me>
Date:   Fri Jun 5 17:24:22 2026 -0300
    Adiciona conteudo no index de outra forma

* commit 95a2e5d8627f0db7086d90b1d7c8b5d219d22 (master)
Author: zelda <vitorveras@proton.me>
Date:   Fri Jun 5 17:23:52 2026 -0300
    Adiciona conteudo no index

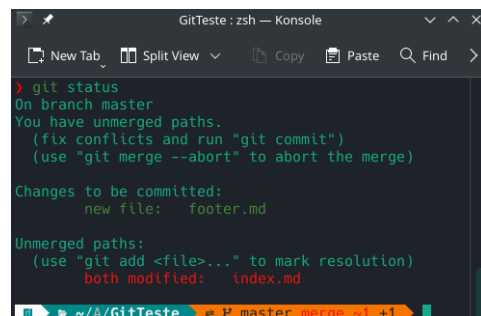
commit 744e095d9259af008a03bf5badedfd759990ecd
Author: zelda <vitorveras@proton.me>
Date:   Fri Jun 5 17:22:59 2026 -0300
    Adiciona index
```

Vamos aplicar o merge da branch `topic` para a branch `master` usando o comando `git merge topic` estando na branch `master` e teremos a seguinte configuração, a partir daí é apresentada a mensagem de conflito:



```
GitTeste: zsh - Konsole
git merge topic
Auto-merging index.md
CONFLICT (content): Merge conflict in index.md
Automatic merge failed. Fix conflicts and then commit the result.
```

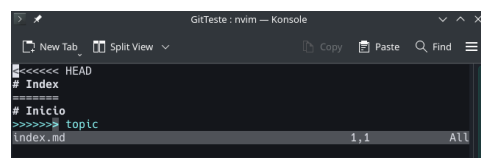
Nesse momento o Git entra em estado de “Merging” e é necessário resolver os conflitos para depois realizar o commit de merge. O Git identifica os arquivos que estão em conflito e marca as alterações usando os marcadores `<<<<<<<< HEAD` para indicar o que atualmente existe e `>>>>>>> <nome>` para indicar o que está vindo do merge, para maiores detalhes dos arquivos podemos usar o comando `git status` para identificar os arquivos em conflito:



```
GitTeste: zsh - Konsole
git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
  new file:   footer.md

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   index.md
```



```
GitTeste: nvim - Konsole
<<<<<< HEAD
# Index
# Inicio
>>>>>> topic
index.md      1,1      All
```

Após a resolução dos conflitos, ou seja, a escolha de quais mudanças devem permanecer, é necessário adicionar os arquivos usando `git add` e depois criar um novo commit usando `git commit` para finalizar o processo de merge. Observe as mudanças com `git log --graph`.

```
GitTeste : git — Konsole
New Tab Split View Copy Paste Find
* commit 29d1424ec49cd506956db7d7c5bf4e4555486b46 (HEAD -> master)
  Merge: 96b3ea5 32b8731
  Author: zelda <vitorveras@proton.me>
  Date:   Fri Jun 5 17:39:37 2026 -0300

      Merge branch 'topic'

* commit 32b87315b30426dfea4ea89a89500c2ee232ed28 (topic)
  Author: zelda <vitorveras@proton.me>
  Date:   Fri Jun 5 17:30:58 2026 -0300

      Adiciona footer

* commit 54ff0fc7103d38d7397706f075df2ec892956b12
  Author: zelda <vitorveras@proton.me>
  Date:   Fri Jun 5 17:24:22 2026 -0300

      Adiciona conteudo no index de outra forma

* commit 96b3ea5a06276bdb7906e96bb147c805d6219d22
  Author: zelda <vitorveras@proton.me>
  Date:   Fri Jun 5 17:23:52 2026 -0300

      Adiciona conteudo no index

* commit 744a905d9259af988a93bf5ba6e8dfd759990ecd
  Author: zelda <vitorveras@proton.me>
  Date:   Fri Jun 5 17:22:59 2026 -0300

      Adiciona index

(END)
```

Rebase

Podemos modificar o histórico (Project History), alterar a ordem de commits, unir commits e modificar mensagens, isso pode ser de certa maneira perigoso pois os commits são a base do controle de versão, eles servem para documentar o processo, um registro cronológico do desenvolvimento.

As diferenças entre merge e rebase

O merge é um processo de integração de branches, ele tem a vantagem de preservar o histórico do desenvolvimento, ou seja, ele mantém a estrutura de árvore dos commits. Dizemos que ele é um processo não destrutivo. Contudo, ele gera um histórico mais complexo, com muitos branches e merges, o que pode dificultar a leitura do histórico.

O rebase é um processo de integração de branches que tem a vantagem de manter um histórico linear, ou seja, ele “rebaseia” os commits da branch atual para cima da branch alvo. Ele é um processo destrutivo.

O uso na prática

A partir de um commit A na branch master criamos uma branch topic e fazemos os commits B, na branch master é realizado um commit C, ou seja, temos a seguinte configuração:

Ao realizar o merge da branch topic para a branch master usando o comando `git merge topic` teremos a seguinte configuração:

```
git log --graph --oneline
* commit f130c7b22312002f8111c45ad07ac38f6629c0c8 (HEAD -> master)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:49 2026 -0300
  Adiciona footer
* commit 2b761994cf8ac3f7b222e773c089b3fe2992fb3e (topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:03 2026 -0300
  Adiciona texto na Index
* commit 6e6e1b0806f493ad0b3c499f2fbff241c926d62d
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:50:00 2026 -0300
```

```
git log --graph --oneline
* commit 71f8f3a4fd4dee8c102a2214202413ff869daf93 (HEAD -> master)
  Merge: f130c7b 2b76199
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:54:55 2026 -0300
  Merge branch 'topic'
* commit 2b761994cf8ac3f7b222e773c089b3fe2992fb3e (topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:03 2026 -0300
  Adiciona texto na Index
* commit f130c7b22312002f8111c45ad07ac38f6629c0c8
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:49 2026 -0300
  Adiciona footer
* commit 6e6e1b0806f493ad0b3c499f2fbff241c926d62d
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:50:00 2026 -0300
  Adiciona index
```

O Git faz o uso da **Recursive Strategy** gerando o commit M que serve apenas para indicar que houve um merge, caso em que algumas vezes esse commit pode ser evitado.

Realize um commit A na branch master, crie uma branch topic e faça o commit B, depois na branch master realize um commit C, ou seja, temos a seguinte configuração:

Ao realizar o rebase da branch topic para a branch master usando o comando `git rebase master` teremos a seguinte configuração:

```
git log --graph --oneline
* commit f130c7b22312002f8111c45ad07ac38f6629c0c8 (HEAD -> master)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:49 2026 -0300
  Adiciona footer
* commit 2b761994cf8ac3f7b222e773c089b3fe2992fb3e (topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:51:03 2026 -0300
  Adiciona texto na Index
* commit 6e6e1b0806f493ad0b3c499f2fbff241c926d62d
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 15:50:00 2026 -0300
```

```
git log --graph --oneline
* commit 27a7032e99b962ba492714895dfc584f9efeb5a1 (HEAD -> topic)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 16:46:40 2026 -0300
  Adiciona texto na Index
* commit 61d7d7fcb0c9e9b1267d11eb40975c46d1948623 (master)
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 16:46:58 2026 -0300
  Adiciona footer
* commit d667df020582b66f921c52e95441dee40ecb3d4
  Author: VerasNp <vitorveras@as>
  Date: Fri Jun 5 16:46:19 2026 -0300
  Adiciona index
```

Dessa maneira ao continuar o desenvolvimento na branch topic podemos usar um merge do tipo **Fast-Forward**.

Isso merece um cuidado pois quando aplicado rebase **os commits terão novos IDs**, assim é preferível evitar:

- Rebase de branches compartilhadas, ou seja, aquelas que já foram enviadas para um repositório remoto e podem ter sido **usadas por outras pessoas**;

Conflitos

Assim como no merge, o rebase também pode gerar conflitos, nesse caso o processo de resolução é o mesmo, ou seja, é necessário resolver os conflitos nos arquivos identificados de sobreposição, contudo com a diferença que ao final fazemos `git rebase --continue` para continuar o processo de rebase.

Undo

Algumas vezes é necessário desfazer mudanças, seja para corrigir um erro ou para refazer um processo de uma maneira diferente. O Git oferece diversas formas de desfazer mudanças, seja em arquivos, commits ou até mesmo branches.

Desfazendo mudanças em arquivos

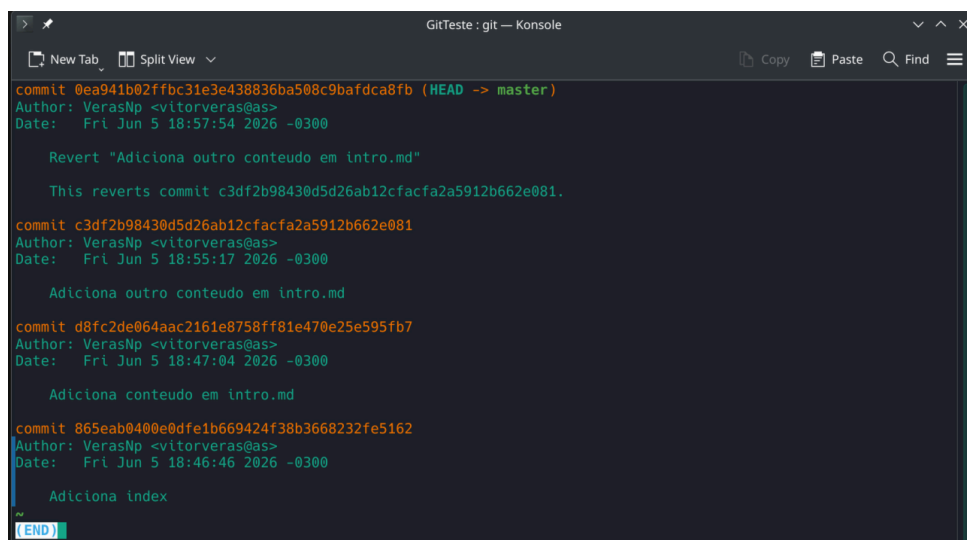
Para desfazer mudanças em arquivos, ou seja, para voltar um arquivo para o estado do último commit, podemos usar o comando `git checkout -- <arquivo>`. Esse comando descarta as mudanças feitas no arquivo até o momento e volta o arquivo para o estado do último commit. Cuidado ao usar esse comando, pois ele descarta as mudanças sem possibilidade de recuperação.

Desfazendo commits

Para desfazer um commit, ou seja, para voltar o estado do repositório para o estado do commit anterior, podemos usar o comando `git reset --hard HEAD~1`. Esse comando descarta o último commit e todas as mudanças feitas nele, voltando o repositório para o estado do commit anterior. Cuidado ao usar esse comando, pois ele descarta as mudanças sem possibilidade de recuperação.

- Para se guiar melhor é possível usar o comando `git reflog` para visualizar onde a HEAD estava até o momento necessário de desfazer o commit ou se preferir usar o hash do commit, basta usar `git reset --hard <hash_commit>`;
- O uso do `--hard` é necessário para descartar as mudanças feitas no commit, caso contrário, as mudanças ficariam no diretório de trabalho e poderiam ser acidentalmente comitadas novamente.

Outra forma de desfazer um commit é usando o comando `git revert <hash_commit>`. Esse comando cria um novo commit que desfaz as mudanças feitas no commit especificado, ou seja, ele mantém o histórico do desenvolvimento, mas desfaz as mudanças feitas no commit especificado. Essa é uma forma mais segura de desfazer um commit, pois ela mantém o histórico do desenvolvimento.



```
GitTeste : git — Konsole
New Tab Split View Copy Paste Find
commit 0ea941b02ffbc31e3e438836ba508c9bafdca8fb (HEAD -> master)
Author: VerasNp <vitorveras@as>
Date: Fri Jun 5 18:57:54 2026 -0300

    Revert "Adiciona outro conteudo em intro.md"

    This reverts commit c3df2b98430d5d26ab12cfacfa2a5912b662e081.

commit c3df2b98430d5d26ab12cfacfa2a5912b662e081
Author: VerasNp <vitorveras@as>
Date: Fri Jun 5 18:55:17 2026 -0300

    Adiciona outro conteudo em intro.md

commit d8fc2de064aac2161e8758ff81e470e25e595fb7
Author: VerasNp <vitorveras@as>
Date: Fri Jun 5 18:47:04 2026 -0300

    Adiciona conteudo em intro.md

commit 865eab0400e0dfe1b669424f38b3668232fe5162
Author: VerasNp <vitorveras@as>
Date: Fri Jun 5 18:46:46 2026 -0300

    Adiciona index

or
(END)
```

GitHub Projects

Integra diferentes pontos de forma gerenciável como Issues, Pull Requests de uma forma gráfica para facilitar as necessidades e processos da equipe.

Nele você pode escolher diferentes templates sendo desde um estilo Kanban onde possibilita um acompanhamento do progresso do trabalho, como também outros templates mais específicos. Por padrão ele também apresenta 3 possibilidades de visualização, por tabela, quadro e roadmap. Futuramente é possível ainda criar novas visualizações.

Configurações

Dentro das configurações de um projeto é possível desde manejar dados mais gerais como nome, descrição e repositório padrão, de onde as issues irão ser importadas de forma automática, até a acessibilidade do projeto com controle de convidados.

Automação

Estando ele ligado a um repositório é possível mover cards automaticamente entre colunas baseado em eventos (abertura, fechamento de Issues/PRs) por exemplo, o que também acaba por facilitar muito ao retirar parte de um trabalho manual